Object-Oriented Programming: The 4 Pillars



KEY IDEAS

Pillar	Idea	Python tools
Encapsulation	bundle data + behaviour, hide internals	`_x`, `__x`, `@property`
Abstraction	expose *what*, hide *how*	`ABC`, `@abstractmethod`, `Protocol`
Inheritance	reuse and specialise a parent	`class B(A)`, `super()`, MRO
Polymorphism	same call, different behaviour	overriding, duck typing, dunders

Access conventions: `x` public; `_x` internal (convention only); `__x` name-mangled to `_Class__x` (avoids subclass clashes, not real privacy).

> **TIP** **Duck typing:** if it has the method, it works. No shared base class needed ("if it quacks like a duck...").

EXAMPLE

```python
class Account:                              # encapsulation
    def __init__(self, balance):
        self.__balance = balance            # name-mangled
    def deposit(self, amt):
        if amt <= 0:
            raise ValueError("amount must be > 0")
        self.__balance += amt
    @property
    def balance(self):
        return self.__balance

acc = Account(100)
acc.deposit(50)
print(acc.balance)                          # → 150
print(acc._Account__balance)                # → 150 (mangled)
```

```python
from abc import ABC, abstractmethod

class Shape(ABC):                           # abstraction
    @abstractmethod
    def area(self): ...

class Circle(Shape):
    def __init__(self, r): self.r = r
    def area(self): return 3.14159 * self.r ** 2

class Square(Shape):
    def __init__(self, s): self.s = s
    def area(self): return self.s ** 2

for s in (Circle(1), Square(2)):            # polymorphism
    print(s.area())                         # → 3.14159 then 4
# Shape() → TypeError: Can't instantiate abstract class
```